Documentation d'installation de Lutim sous Debian 12

Sommaire :

1. Présentation 3
1. Objectif et périmètre 3
2. Pré-requis (root) 3
3. Créer l'utilisateur applicatif et cloner le code (root) 3
4. Ouvrir une session PROPRE de l'utilisateur lutim 4
5. Créer un environnement Perl local SAIN (lutim) 4
6. Installer les dépendances du projet (lutim) 5
7. Configurer Lutim (lutim) 5
8. Préparer les répertoires et permissions (root) 6
9. Démarrer Lutim (session propre lutim) 6
10. Accès au service 6
11. Vérifications rapides 7
12. Conclusion 7

1. Présentation

Lutim (Let's Upload That Image) est une application web libre écrite en Perl.

Elle permet d'héberger et de partager des images en ligne.

Autrefois utilisée par Framapic, un service de Framasoft fermé en 2021, Lutim reste un logiciel libre, maintenu et installable sur un serveur Debian.

Cette documentation décrit comment installer et configurer Lutim sous Debian 12 de manière stable et sécurisée.

1. Objectif et périmètre

Mettre en service Lutim (Let's Upload That Image) sur Debian 12, accessible sur le port 8080 de la machine. Nous n'abordons pas ici Nginx/HTTPS ni systemd pour rester focalisés sur un service opérationnel minimal.

2. Pré-requis (root)

Connectez-vous en root et installez les paquets nécessaires :
Dans cette première étape, nous préparons le système pour garantir une base saine : mise à jour des paquets, correctifs de sécurité et installation des dépendances nécessaires à la compilation et à l'exécution de Lutim.

```
apt update && apt upgrade -y
```

Cette commande met à jour la base des paquets disponibles puis installe automatiquement toutes les mises à jour du système. C'est une bonne pratique de démarrer toute installation par un système à jour.

```
apt install -y build-essential make gcc git imagemagick libimage-magick-perl \
  curl vim pkg-config libssl-dev zlib1g-dev libexpat1-dev cpanminus libpq-dev
```

Nous installons ici : les outils de compilation C/C++ (build-essential, gcc, make), Git pour récupérer le code, ImageMagick et son module Perl pour le traitement d'images, ainsi que des bibliothèques de développement (SSL, zlib, expat) et cpanminus pour gérer les modules Perl. libpq-dev est utile si vous utilisez PostgreSQL plus tard.

3. Créer l'utilisateur applicatif et cloner le code (root)

Nous isolons l'application derrière un utilisateur dédié nommé « lutim ». Cela limite l'impact en cas de problème et clarifie les permissions.

```
useradd -r -m -d /var/www/lutim -s /bin/bash lutim
```

Création d'un compte système (-r) avec un dossier personnel à /var/www/lutim et un shell Bash. Le répertoire contiendra les fichiers de l'application et son environnement local Perl.

```
passwd lutim    # choisissez un mot de passe (ex: lutim)
```

Définissez un mot de passe afin de pouvoir ouvrir une session interactive ponctuelle si nécessaire.

```
cd /var/www
```

On se place dans le répertoire parent où sera stockée l'application.

```
rm -rf lutim            # si un ancien dossier existe
```

Si une ancienne installation est présente, on la supprime pour repartir sur une base propre.

```
git clone https://framagit.org/luc/lutim.git
```

On récupère le code source officiel de Lutim depuis le dépôt Framagit, ce qui crée le dossier /var/www/lutim.

```
chown -R lutim:lutim /var/www/lutim
```

On attribue la propriété des fichiers à l'utilisateur applicatif afin qu'il puisse installer ses dépendances et écrire ses logs.

4. Ouvrir une session PROPRE de l'utilisateur lutim

Pour éviter toute pollution d'environnement (variables héritées de root), on ouvre une session « propre ».

```
env -i su -s /bin/bash lutim
```

Cette commande démarre une session pour l'utilisateur lutim avec un environnement minimal (env -i). Cela évite les erreurs du type « /root/perl5/... Permission non accordée ».

5. Créer un environnement Perl local SAIN (lutim)

Nous allons préparer un Perl « local::lib » propre à l'utilisateur afin d'isoler les modules de l'application du Perl système.

```
rm -rf ~/perl5
```

On supprime tout environnement Perl utilisateur préexistant pour éliminer d'anciens modules corrompus ou incompatibles.

```
rm -f ~/.bashrc
```

On repart d'un .bashrc vierge pour contrôler précisément ce qui s'y trouve.

```
echo '# Environnement Perl local (lutim)' > ~/.bashrc
echo 'eval "$(perl -I ~/perl5/lib/perl5 -Mlocal::lib=~/perl5)"' >> ~/.bashrc
echo 'export PATH="$HOME/perl5/bin:$PATH"' >> ~/.bashrc
```

Ces lignes configurent l'environnement pour utiliser un répertoire local de modules Perl (~/.perl5) et ajoutent son dossier bin au PATH. Ainsi, les outils installés avec cpanm seront accessibles.

```
source ~/.bashrc
```

On recharge le profil pour prendre en compte immédiatement les variables définies.

```
cpanm --notest --local-lib=~/perl5 Carton
```

On installe Carton localement. Carton est l'outil qui gère les dépendances du projet à partir du fichier cpanfile.

```
which carton   # doit afficher /var/www/lutim/perl5/bin/carton
```

Cette vérification s'assure que le binaire carton présent dans ~/perl5/bin est bien utilisé.

6. Installer les dépendances du projet (lutim)

Nous allons maintenant installer toutes les dépendances Perl spécifiques à Lutim, telles que définies par le projet.

```
cd /var/www/lutim
```

On se place à la racine du projet Lutim.

```
carton install
```

Carton lit le fichier cpanfile et installe localement l'ensemble des modules requis dans ~/perl5.

```
carton exec cpanm --notest DBD::Pg
```

```
lutim@debian:~$ carton exec cpanm --notest DBD::Pg
DBD::Pg is up to date. (3.18.0)
lutim@debian:~$
```

Si le module DBD::Pg (pilote PostgreSQL) manque, cette commande le force à être installé dans l'environnement géré par Carton.

7. Configurer Lutim (lutim)

On part du modèle de configuration fourni par le projet, puis on l'adapte à nos besoins.

```
cd /var/www/lutim
cp lutim.conf.template lutim.conf
nano lutim.conf
```

Le fichier suivant est un **exemple minimal fonctionnel** :

```
{
  hypnotoad => {
    listen => ['http://*:8080'],
    pid_file => '/var/www/lutim/lutim.pid',
  },
  contact => 'bernard@thiery.com',
  max_file_size => 20000000,
  enable_crypt => 1,
  allow_public_uploads => 1,
  delete_no_longer_viewed => 0,
```

```
  allow_zip_download => 1,
  data_dir => '/var/www/lutim/files',
  log_dir  => '/var/www/lutim/log',
}
```

Ici, nous indiquons à Hypnotoad (serveur Mojo) d'écouter sur le port 8080 toutes interfaces, nous définissons un contact administrateur, une taille maximale d'upload, et les répertoires de données et de logs.

8. Préparer les répertoires et permissions (root)

Les répertoires de stockage (fichiers et logs) doivent exister et appartenir à l'utilisateur applicatif.

```
exit
```

On quitte la session de l'utilisateur lutim pour repasser root et créer/attribuer les répertoires.

```
mkdir -p /var/www/lutim/files
mkdir -p /var/www/lutim/log
touch /var/www/lutim/lutim.pid
chown -R lutim:lutim /var/www/lutim
```

Nous créons les dossiers de données et de logs, un fichier PID pour Hypnotoad, puis nous donnons la propriété complète à l'utilisateur lutim.

9. Démarrer Lutim (session propre lutim)

Nous redémarrons une session propre de l'utilisateur pour lancer Lutim via Hypnotoad sous l'environnement Carton.

```
env -i su -s /bin/bash lutim
source ~/.bashrc
cd /var/www/lutim
carton exec hypnotoad script/lutim
```

À ce stade, vous devriez voir un message de démarrage similaire à :

```
lutim@debian:~$ carton exec hypnotoad script/lutim
[2025-10-24 13:11:27.03066] [23140] [info] Listening at "http://*:8080"
Web application available at http://127.0.0.1:8080
```

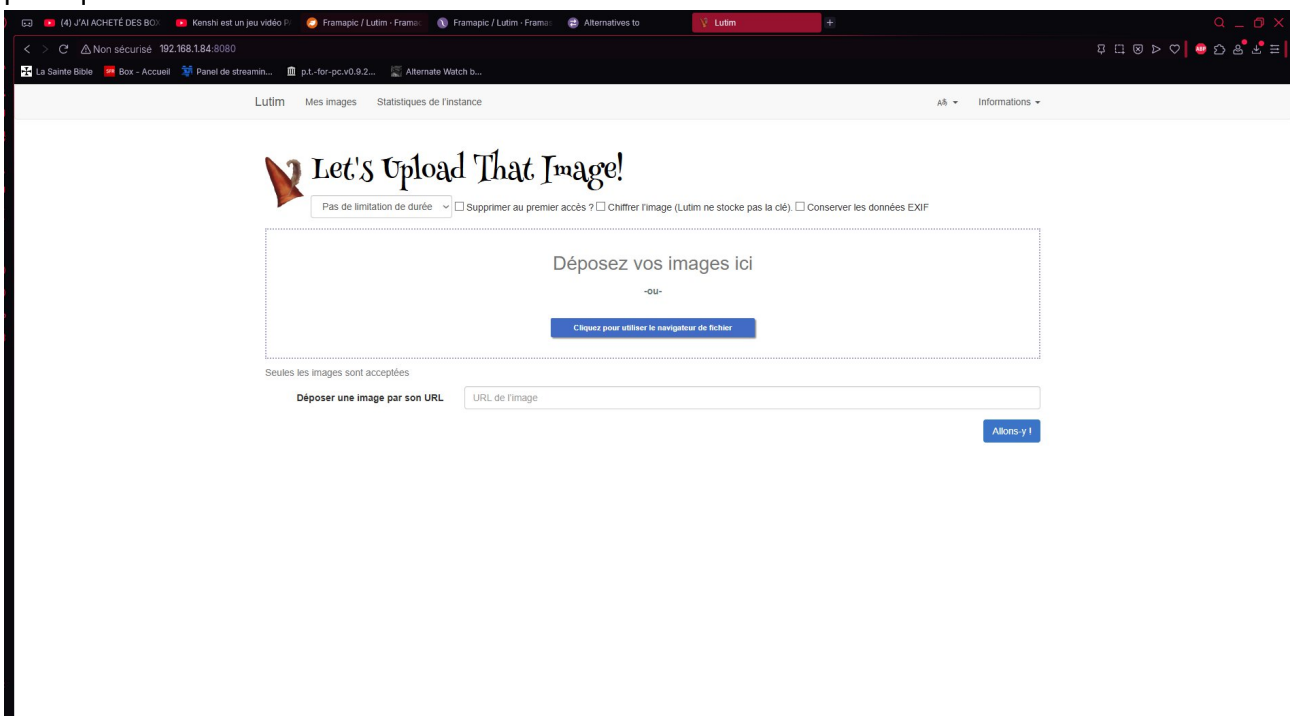
Cela indique que le service écoute bien sur le port 8080 et que l'applicatin est accessible localement.

10. Accès au service

Depuis un poste du réseau local (ou la VM elle-même), ouvrez votre navigateur et saisissez l'URL :

```
http://<IP_DE_LA_VM>:8080  (ex. http://192.168.1.84:8080)
```

Si vous êtes en environnement virtualisé, utilisez le mode réseau « Accès par pont (Bridged) » pour que la VM obtienne une adresse IP du LAN.



Et voilà, il est désormais possible de déposez des images directement ici.

11. Vérifications rapides

Nous validons que le port 8080 est effectivement à l'écoute, puis nous ouvrons le pare-feu si nécessaire.

```
ss -ltnp | grep 8080
```

Cette commande liste les sockets TCP en écoute et filtre sur le port 8080 pour confirmer que Hypnotoad est bien lancé.

```
ufw allow 8080/tcp && ufw reload
```

12. Conclusion

Lutim est désormais installé et opérationnel sur Debian 12, avec un environnement Perl local isolé et sain. Cette procédure pas à pas vous permet d'obtenir un service fonctionnel sur le port 8080, prêt pour une intégration ultérieure derrière un reverse-proxy (Nginx) et l'activation de HTTPS